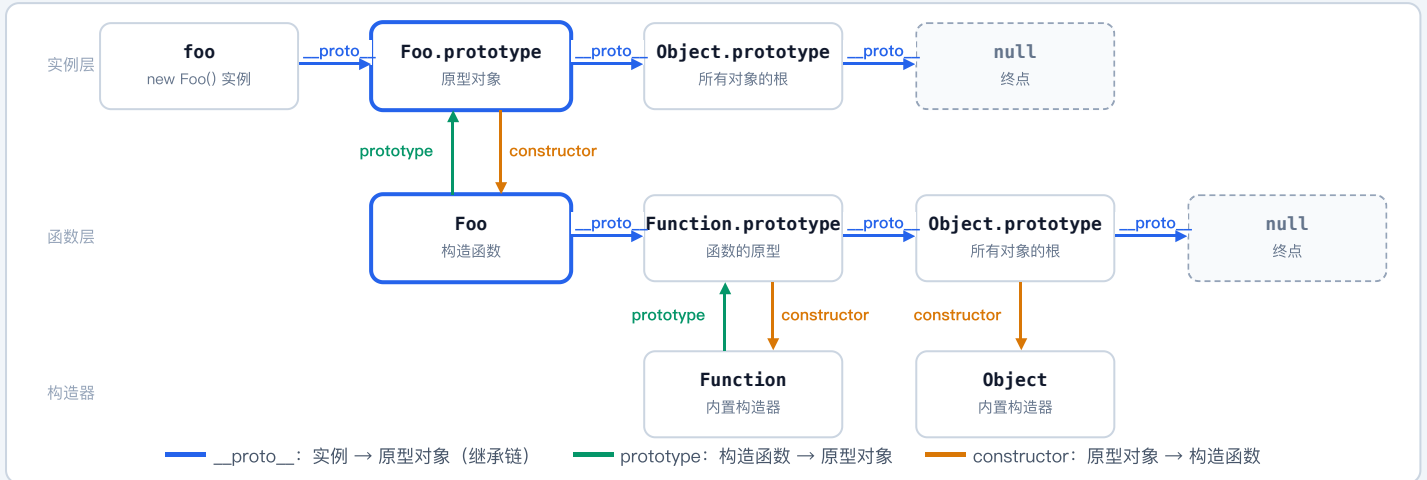


原型链全景速查

一切皆对象：属性查找沿 `__proto__` 逐级向上直到 `null`；只有函数拥有 `prototype`，所有对象都有 `__proto__`。

```
function Foo() {} · const foo = new Foo()
```

全景图 实例链 / 函数链 / 构造器三层，箭头颜色见下方图例



三条黄金规则

- 实例.__proto__ === 构造函数.prototype —— 实例通过 __proto__ 指向其构造函数的 prototype
- 构造函数.prototype.constructor === 构造函数 —— 原型对象自带 constructor 反向引用
- 一切函数 (含 Object) 的 __proto__ 都是 Function.prototype；所有链的终点都是 Object.prototype → null

new 做了 4 步

- 创建空对象 obj
- 连接原型 obj.__proto__ = Foo.prototype
- 绑定 this Foo.call(obj) 并执行函数体
- 返回 obj；若 Foo 显式 return 对象则返回该对象

```
function myNew(Ctor, ...args) {  
  const obj = Object.create(Ctor.prototype) // ① ②  
  const ret = Ctor.apply(obj, args) // ③ this → obj  
  return ret && typeof ret === 'object' ? ret : obj // ④  
}
```

instanceof 原理

语义：右侧 `Ctor.prototype` 是否在左侧 `obj` 的原型链上 —— 沿 `__proto__` 逐级比较，直到 `null`。

```
function myInstanceOf(obj, Ctor) {  
  let p = Object.getPrototypeOf(obj)  
  while (p !== null) {  
    if (p === Ctor.prototype) return true  
    p = Object.getPrototypeOf(p)  
  }  
  return false  
}
```

走一遍：foo instanceof Object → foo → Foo.prototype → Object.prototype 命中 → true ✓

面试快问快答

问题	答案
foo.constructor 为什么是 Foo?	自身无 constructor，沿 __proto__ 在 Foo.prototype 上找到 (Foo.prototype.constructor === Foo)
为什么 foo instanceof Object 是 true?	Object.prototype 在 foo 的原型链上 (foo → Foo.prototype → Object.prototype)
Function.__proto__ 为什么 === Function.prototype?	Function 由引擎自举 (bootstrapped)，特殊规则
Object.__proto__ 是什么?	Function.prototype —— Object 也是函数
foo.prototype 是什么?	undefined —— 只有函数才有 prototype 属性

高频坑位

- prototype 与 __proto__ 混淆：prototype 只有函数有 (放共享方法)；__proto__ 每个对象都有 (指向构造函数的 prototype)
- Object.create(null) 无任何原型链，连 toString / hasOwnProperty 都没有
- 构造函数显式 return 对象时，new 返回该对象；return 基本类型则被忽略，仍返回新对象
- 重写 Foo.prototype 后，旧实例仍指向旧原型；新原型丢失 constructor，需手动补回